

基于“假设-验证”闭环的 RCA Agent 实践

刘至浩

趣丸科技 高级平台研发工程师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 困境与挑战

02 设计与实践

03 案例展示

04 思考与总结

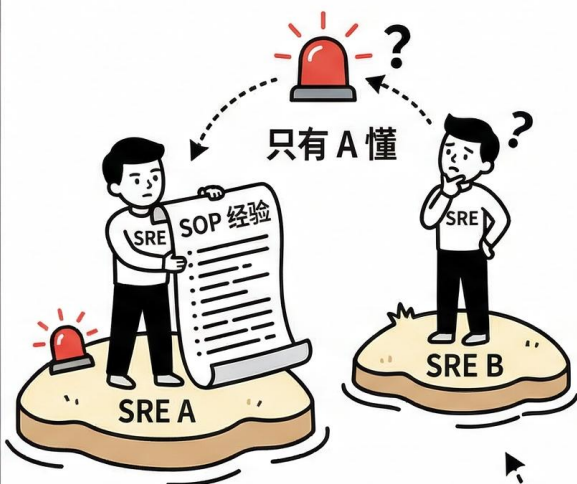
05 未来展望

01

困境与挑战

需求背景

1. 打破知识孤岛



现状：A配的告警，只有A懂。
B收到是无效信息。

目标：经验全员可复用

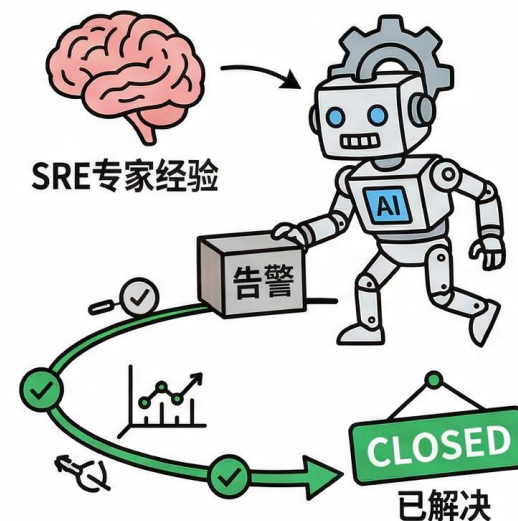


2. 守护深夜尊严



诉求：不开电脑，让AI先跑排查。
手机看影响范围、异常指标。
确定需不需要起床。

3. 转化经验为执行力



提效：SOP变成Agent跑腿指令。
结果：AI自主分析、定位、跟踪，
直至闭环结案。

理想的样子

结果导向

拒绝无效拉扯，目标是一次交互出结论，不让用户陪着 AI 做填空题。

快速排查

比“人”快，多方向思考与验证。



严谨可靠

坚持“没证据就不乱说”，用证据说话。

自我进化

通过持续的系统反馈闭环，不断沉淀总结、自我进化，像“老司机”。

现实的样子



容易幻觉，结果不稳定

排障时会“脑补”出看似合理实则完全错误的根因结论。



经验工程化

专家经验难提取，多轮后导致模型逻辑“失焦”。



工具和系统集成麻烦

返回数据二次加工，工具集相互影响，跨团队协作与工具对齐成本高。



长链路时很脆弱

一旦步骤变多、任务变复杂，成功率会明显下降，长链路自主执行尤其脆弱。

生产很难跑“稳”，用户很难“买单”

Agent 层面

在根因定位场景下

- Context 中 Prompt “打架”
- 2-3轮后，成功率骤减
- 低效信息噪音多
- 优化难，常常要追踪“模型输出 + 工具调用 + 重试 + 上下文变化”



一旦 Observation 带了噪声，整个 Plan 就会很容易跑偏。

工具缺陷

“可观测” 对AI不友好

- Schema 过于繁琐
- 观测数据的高维与冗余导致响应体（Payload）通胀，极易触碰 LLM 的上下文窗口极限

——Tool call——>

```
{
  "metric":
  "node_namespace_pod_container:container_cpu_usage_seconds_total:sum_rate",
  "labels": {
    "cluster": "cluster-a",
    "container": "sandbox",
    "namespace": "sre-platform",
    "node": "10.xx.xxx.xx",
    "pod": "service-A-6c8d974dd4-xwljj",
    "prometheus": "monitoring/k8s"
  }
}
```

——API——>

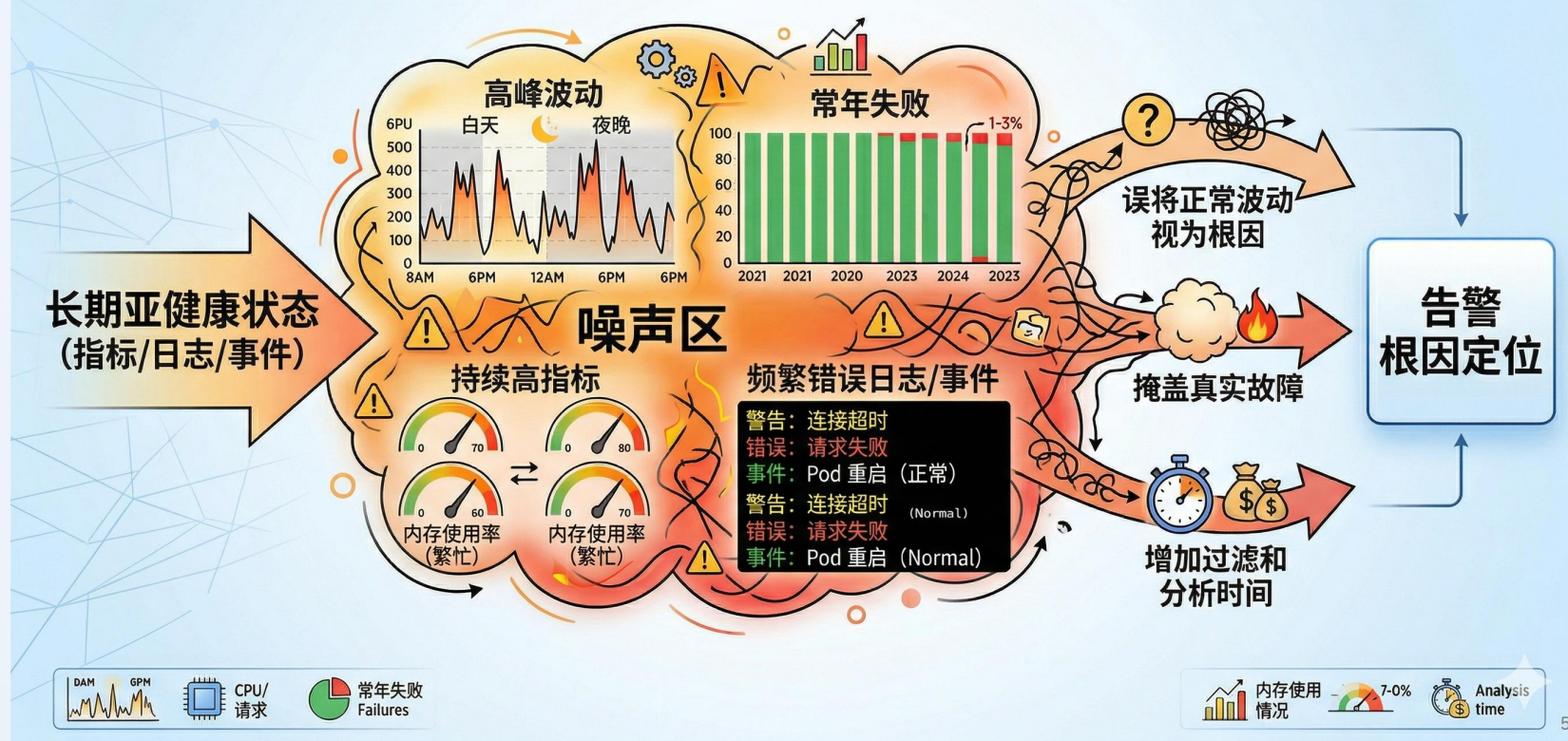
```
{
  "code": 0,
  "data": {
    "result": [
      {
        "metric": {
          "cluster": "cluster-a",
          "namespace": "sre-platform",
          "pod": "service-A-6c8d974dd4-xwljj",
          "workload": "solo-center",
          "workload_type": "deployment"
        },
        "values": [
          [
            1773214830,
            "0.075"
          ],
          [
            1773214860,
            "0.079"
          ],
          [
            1773214890,
            "0.08"
          ],
          [
            1773214920,
            "0.079"
          ],
          [
            1773214950,
            "0.08"
          ],
          [
            1773214980,
            "0.083"
          ],
          [
            1773215010,
            "0.076"
          ],
          [
            1773215040,
            "0.078"
          ],
          [
            1773215070,
            "0.079"
          ],
          [
            1773215100,
            "0.083"
          ],
          [
            1773215130,
            "0.077"
          ]
        ]
      }
    ]
  },
  "msg": "操作成功"
}
```

Agent 往往面临 “心有余而力不足”

服务亚健康状态

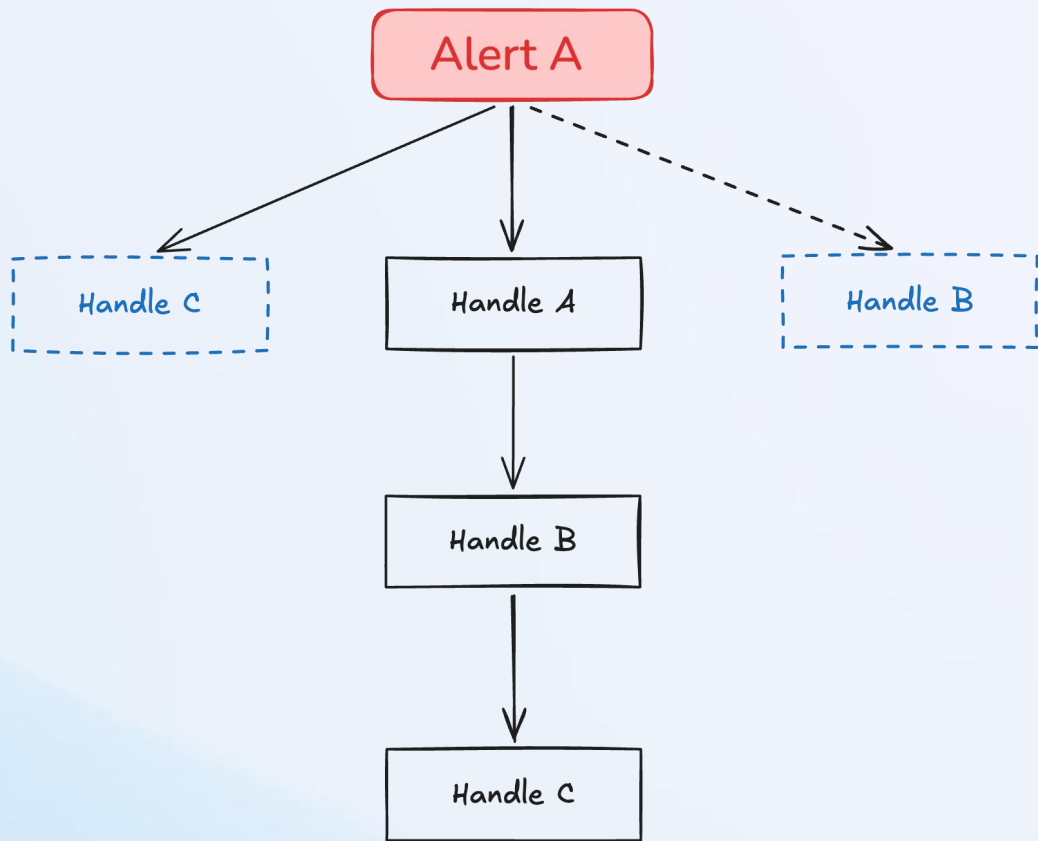
- 变更中、干预中
- 业务周期性噪声
- 导致 Agent 提前结束

长期亚健康服务作为“噪声”对根因分析的挑战



每个资源有其“画像”，多轮分析后会导致Agent“水土不服”

用户层面



“并行排查，提升效率。”

🕒 恢复时间: 2026-03-10 21:40:36 +0800 CST

🕒 发送时间: 2026-03-10 21:41:23 +0800 CST

📄 告警内容:

集群 [redacted] 网关[istio-ingressgateway-tt-api] 请求服务 [online-logic] 的GRPC调用 QPS 持续3分钟 ≥ 2 (当前值3373.30)

集群 [redacted] 网关[istio-ingressgateway-tt-a] 请求服务 [online-logic] 的GRPC调用 P99 持续3分钟 $\geq 2000\text{ms}$ (当前值1491.93ms)

[查看应用监控](#) [查看告警规则](#)

[告警排查](#) 告警根因定位准确性反馈 ▼

告警处理过程:

2026-03-10 21:35:22 启动告警rca根因定位

2026-03-10 21:37:42 结果: 问题根因是服务调用链路中, 中间件 (Redis数据库) 不可用。其中, Redis在异常期间的CPU使用率高达95%, 连接数超配额, 导致部分外部接口出现502错误响应, 进而引发服务可用率骤降至83%, 远低于正常水平。

详细报告链接: [内网地址](#) | [外网地址](#)

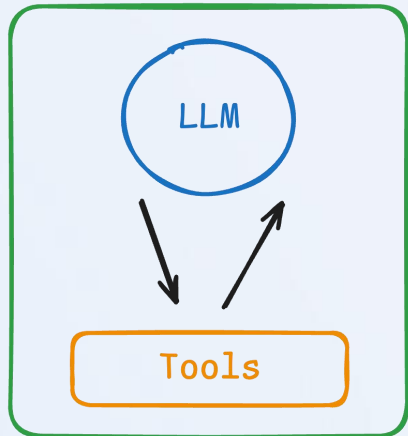
2026-03-10 21:37:42 结果: 成功

“没有证据支撑的结论，比没有结论更可怕。”

02

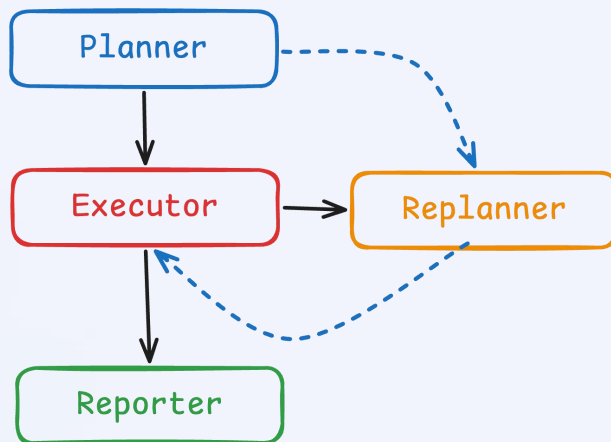
设计与实践

Agent 实践



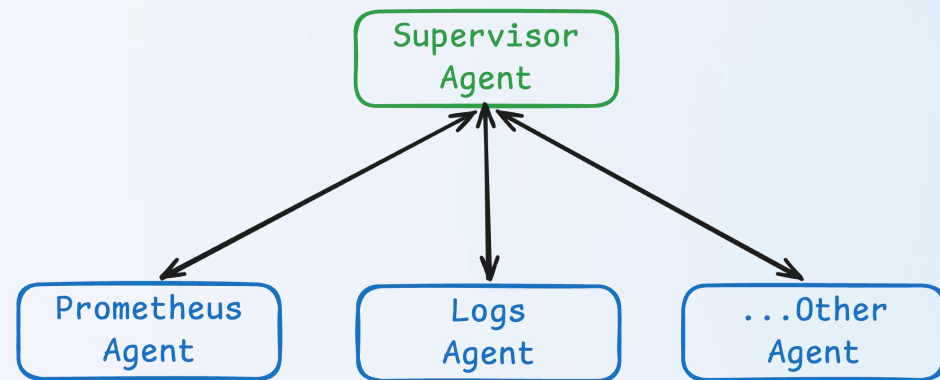
ReAct

- 容易在长任务中产生“幻觉”或死循环。
- Token 消耗随步骤增加而剧增。
- 缺乏全局观。
- 简单、交互性强、具有动态适应性。



Plan-and-Execute

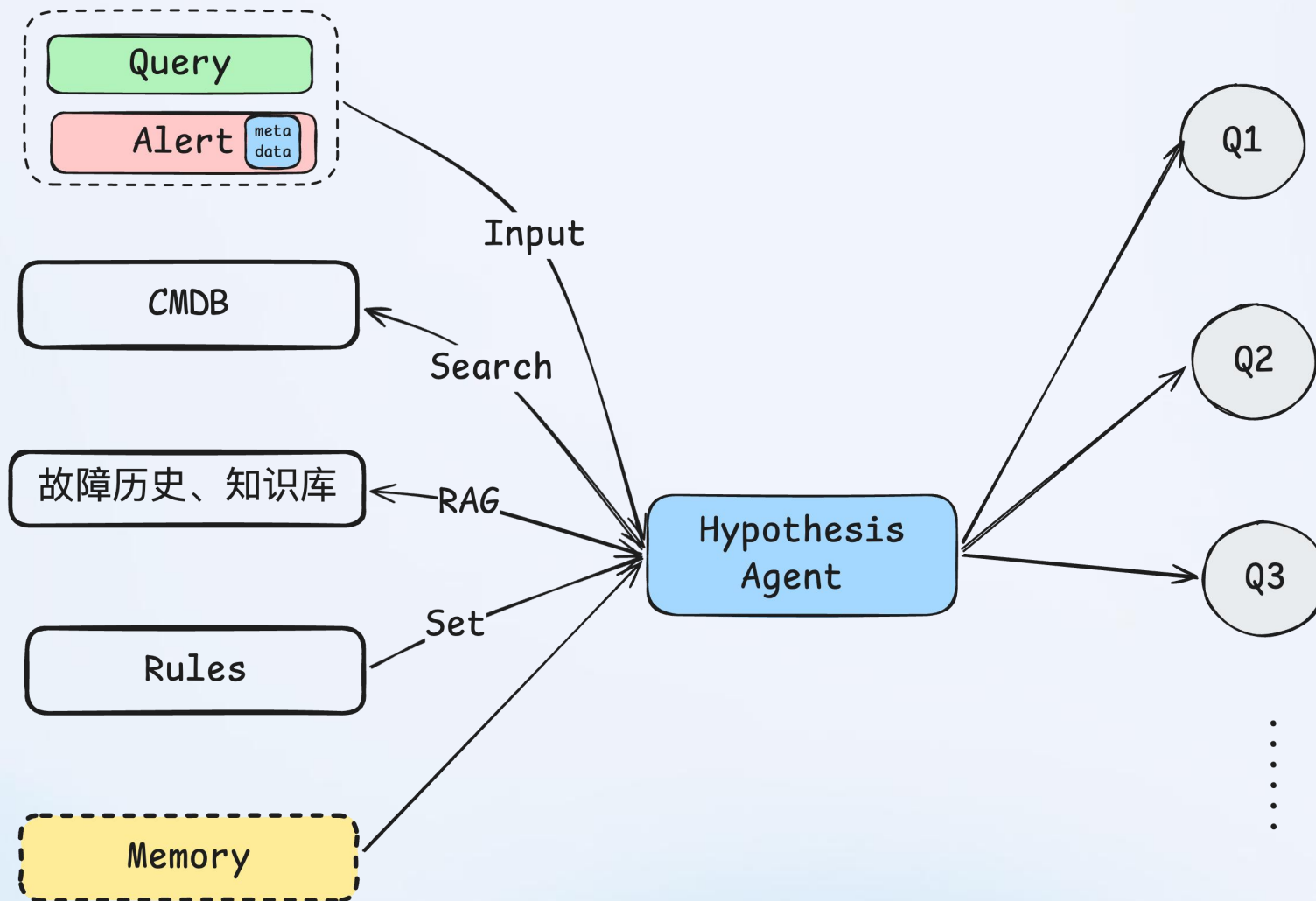
- 初始计划如果出错可能导致全盘失败。
- 需要复杂的重计划机制。
- 整体响应速度慢。
- 步骤相对固定的复杂任务（如数据分析、报告生成）。



Multi-Agent

- 多轮会有噪音。
- 智能体间通信与协调成本高昂。
- “责任推诿”，调试难度大。
- 适合边界明确，协商推理场景。

Hypothesis Agent



Verification Agent Prompting

Text 1: Charlie is rough. Charlie is white. Erin is blue. Erin is rough. Erin is young. Gary is blue. Gary is green. Gary is quiet. Gary is white. Harry is blue. Harry is rough. All rough, white things are green. All young, blue things are quiet. If something is young then it is blue. White, blue things are quiet. If something is green then it is young. Quiet, green things are cold. All green, cold things are rough.
Question: Harry is cold. **Answer:** True

Text 2: Bob is rough. Fiona is not furry. Fiona is white. Young, nice things are big. Blue things are big. If something is young and white then it is nice. All nice things are young. All white things are young. If something is blue and not big then it is not rough.

Question: Bob is not young.
Answer: Unknown

Standard Prompting

Model Input

Q: Judge the following conclusion 'Harry is cold.' is correct, wrong, or unknown, based on the following texts: **Text 1**

A: The conclusion 'Harry is cold.' is True.

Q: Judge the following conclusion 'Bob is not young.' is correct, wrong, or unknown, based on the following texts: **Text 2**

Model Output

Q: The conclusion 'Bob is not young.' is True.

Chain-of-Thought Prompting

Model Input

Q: Judge the following conclusion 'Harry is cold.' is correct, wrong, or unknown, based on the following texts: **Text 1**

A: Start from 'Harry is quiet' and 'Quiet things are nice'. Then can know 'Harry is nice'. And 'All nice things are cold' is given. Finally it can be deduced 'Harry is cold'. So the conclusion 'Harry is cold.' is True.

Q: Judge the following conclusion 'Harry is not quiet.' is correct, wrong, or unknown, based on the following texts: **Text 2**

Model Output

Q: Start from 'Bob is rough' and 'If something is blue and not big then it is not rough'. Then can know 'Bob is not blue'. And 'All white things are young' is given. Finally it can be deduced 'Bob is not young'. So the conclusion 'Bob is not young.' is True.

Hypothesis Testing Prompting

Model Input

Q: Judge the following conclusion 'Harry is cold.' is correct, wrong, or unknown, based on the following texts: **Text 1**

A: First assume the conclusion is True and start from Harry is cold. Because the rules state that all nice things are cold, correct conclusion needs to be satisfied that Harry is nice. And the rules states that quiet things are nice. It needs to be satisfied Harry is quiet, which is also stated in the facts. So the conclusion 'Harry is cold.' is True.

Q: Judge the following conclusion 'Bob is not young.' is correct, wrong, or unknown, based on the following texts: **Text 2**

Model Output

Q: First assume the conclusion is True and start from Bob is not young. Because the rules state that all white things are young, correct conclusion needs to be satisfied that Bob is not white. And there are no facts or rules about 'not white'. Then assume the conclusion is False and start from Bob is young. Because the rules state that all nice things are young, correct conclusion needs to be satisfied that Bob is nice. And there are no facts or rules about 'nice'. So the conclusion 'Bob is not young.' is Unknown.

Text 1

事实: 磁盘已满。

规则: 磁盘满属于“严重故障”。所有“严重故障”都会触发报警。

问题: 当前是否触发报警?

答案: 正确 (True) (逻辑闭环)

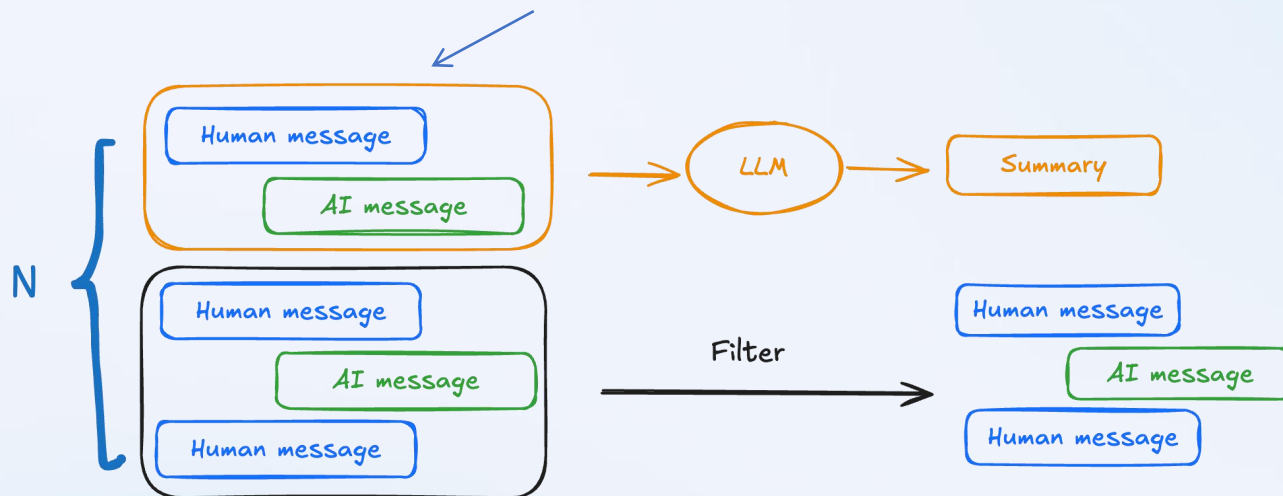
Text 2

事实: 内存升高。

规则: 磁盘满属于“严重故障”。所有“严重故障”都会触发报警。

问题: 当前是否触发报警?

答案: 未知 (Unknown) (逻辑缺失)



<https://arxiv.org/abs/2405.06707>

Verification Agent Prompting

标准 (Standard Prompting)

Model Input (示例 Prompt):

Q: 根据 Text 1~N, 判断结论 ‘正在报警’ 是否正确? }_N

A: 结论是 正确 (True)。

Q: 根据 Text 2, 判断结论 ‘正在报警’ 是否正确?



A: 结论是 **正确 (True)**。

模型有时会直接凭直觉联想 (内存高 -> 报警)。

思维链 (Chain-of-Thought Prompting)

Model Input (线性推导 Prompt):

Q: 根据 Text 1~N, 判断结论 ‘正在报警’ 是否正确? }_N

A: 从 ‘磁盘已满’ 开始, 由于这是 ‘核心故障’, 而核心故障必报警。所以结论是 True。

Q: 根据 Text 2, 判断结论 ‘正在报警’ 是否正确?



A: 从 ‘内存升高’ 开始。内存高说明系统压力大, 这属于 ‘核心故障’。根据规则, 核心故障必报警。所以结论是 **正确 (True)**。

Input “通过思路去证明 True”, 导致模型私自断言了 “内存高 = 核心故障”。

假设检验 (Hypothesis Testing Prompting)

Model Input (双向验证 Prompt):

Q: 根据 Text 1~N, 判断结论 ‘正在报警’ 是否正确? }_N

A: 首先假设结论为真: 为了满足报警, 需满足核心故障, 即磁盘已满。事实确实如此。所以为 True。

Q: 根据 Text 2, 判断结论 ‘正在报警’ 是否正确?



A: 首先**假设结论为真** (正在报警): 规则要求必须满足 ‘磁盘已满’。但事实是 ‘内存升高’, 没提磁盘。无法证实。

接着**假设结论为假** (没有报警): 这需要证明不满足核心故障。但规则没定义 ‘内存升高’ 是否属于非核心故障。无法证伪。

结论: **未知 (Unknown)**。

Input 里的 “假设检验” 范式强制模型在推导 Text 2 时去寻找必要条件, 从而发现了证据链缺失。

AI 的 “脑补” 比 “不知道” 更危险; 不需要一个 ‘猜对’ 的 AI, 而需要一个 ‘没证据就不乱说’ 的严谨专家。

Verification Agent Tools

工具过滤+命名+工件

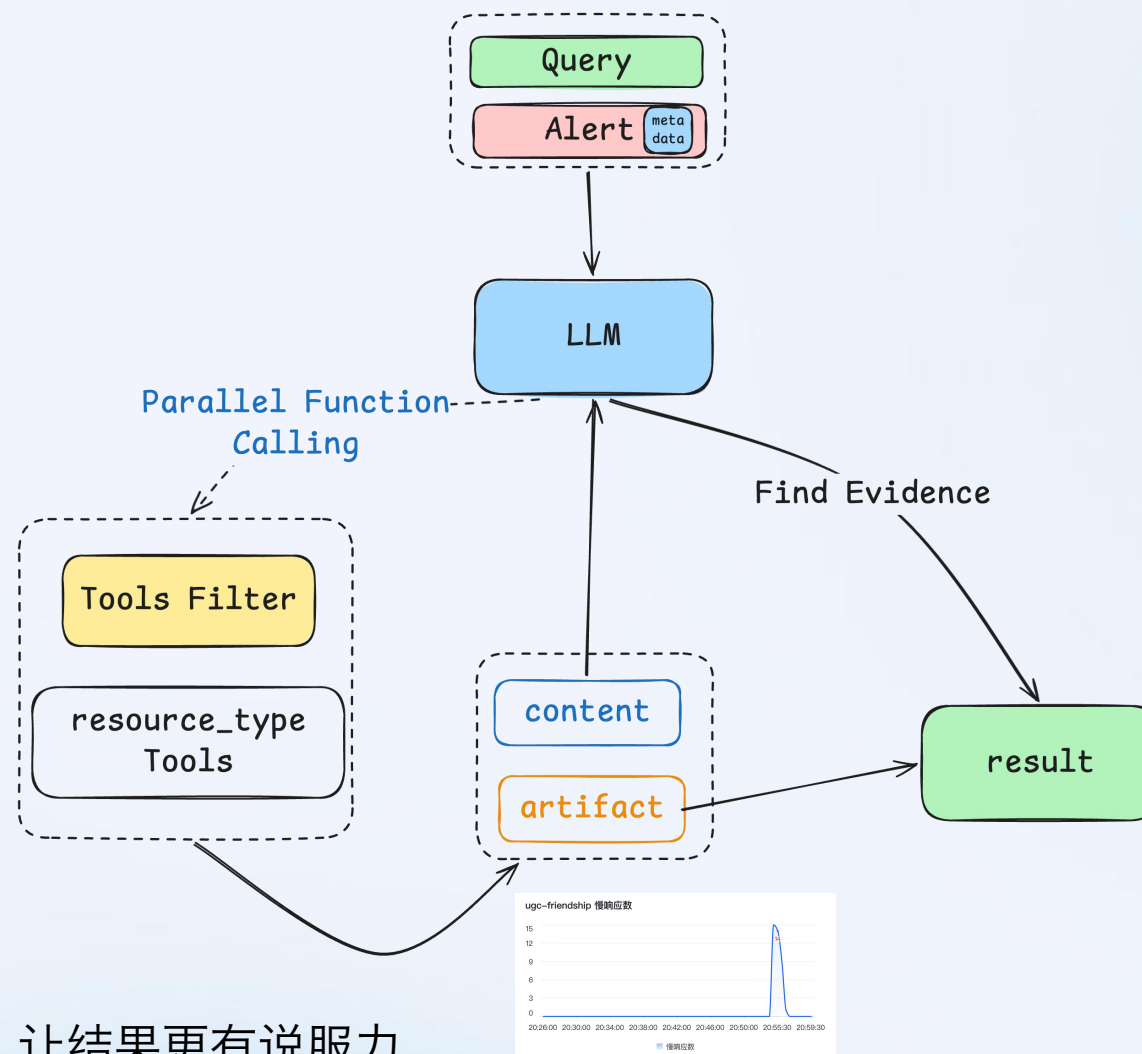
工具名称	功能描述	返回类型
application_metric_interface_latency_is_abnormal	检测接口 P99 延迟是否异常 (多接口曲线)	content + artifact
application_metric_interface_error_rate_is_abnormal	检测接口错误率是否异常	content + artifact
application_metric_interface_qps_is_abnormal	检测接口 QPS 是否异常	content + artifact
application_metric_interface_slow_response_is_abnormal	检测接口慢响应是否异常	content + artifact
application_metric_instance_latency_is_abnormal	检测实例延迟是否异常	content + artifact
application_metric_instance_error_rate_is_abnormal	检测实例错误率是否异常	content + artifact
application_metric_instance_qps_is_abnormal	检测实例 QPS 是否异常	content + artifact
application_metric_instance_slow_response_is_abnormal	检测实例慢响应是否异常	content + artifact
application_metric_latency_is_abnormal	检测应用概览延迟	content + artifact
application_metric_error_rate_is_abnormal	检测应用概览错误率	content + artifact
application_metric_qps_is_abnormal	检测应用概览 QPS	content + artifact
application_metric_slow_response_is_abnormal	检测应用慢响应	content + artifact

日志分析工具

工具名称	功能描述	返回类型
application_log_error	获取应用错误日志	content + artifact
application_log_trend_is_abnormal	检测日志趋势异常 (统计方法检测突增)	dict
logs_is_abnormal	日志短链接查询 (解析短链接提取关键日志)	dict

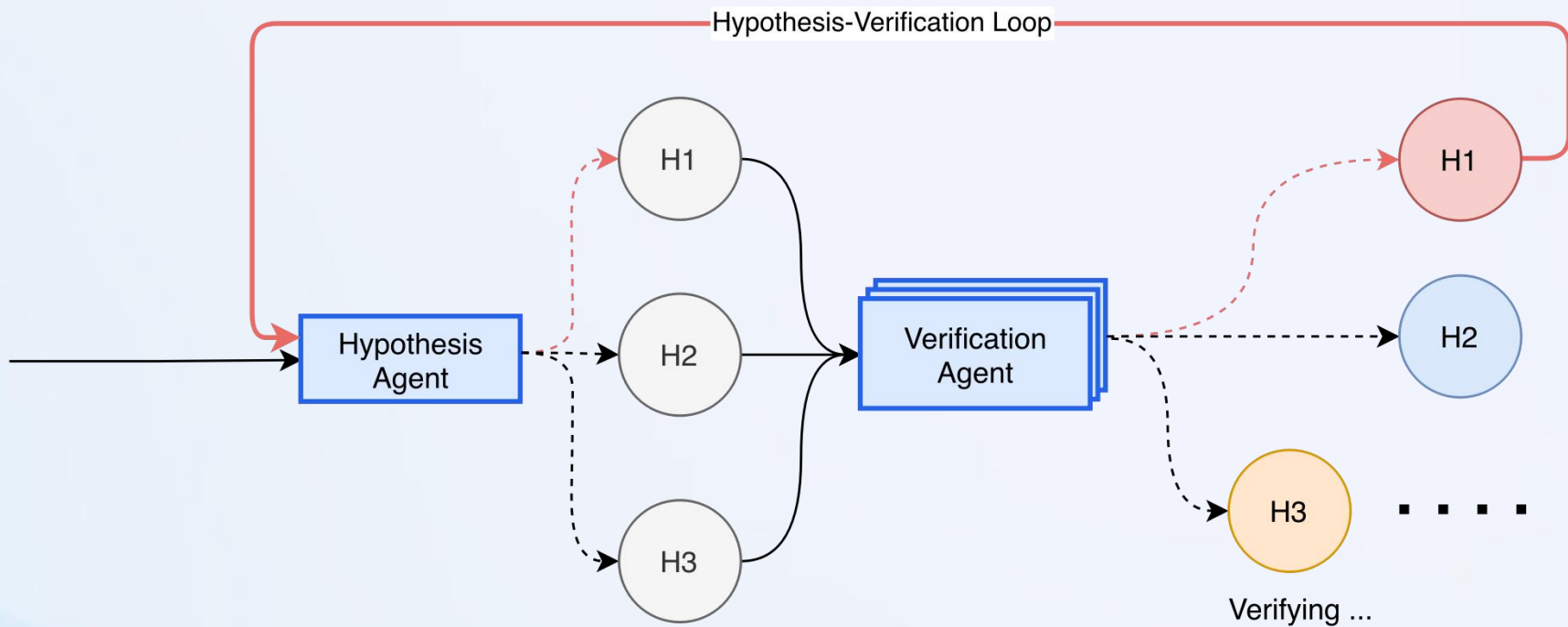
Kubernetes 工具

工具名称	功能描述	返回类型
kubernetes_pod_metric_cpu_is_abnormal	检测 Pod CPU 使用量 (含 Request/Limit 基线)	content + artifact
kubernetes_pod_metric_memory_is_abnormal	检测 Pod 内存使用量 (含 Request/Limit 基线)	content + artifact
kubernetes_pod_event_is_abnormal	检测 Pod 事件异常	content + artifact



图片比文字更直观，让结果更有说服力

Hypothesis-Verification Loop



当所有叶子节点均被证伪，或推理达到设定层级 -> 收敛总结

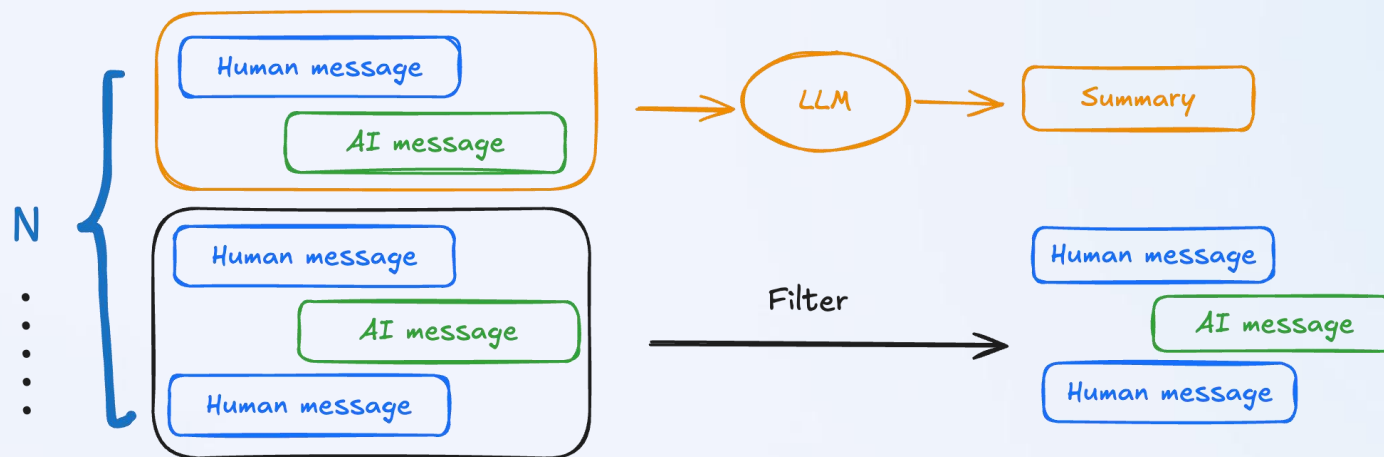
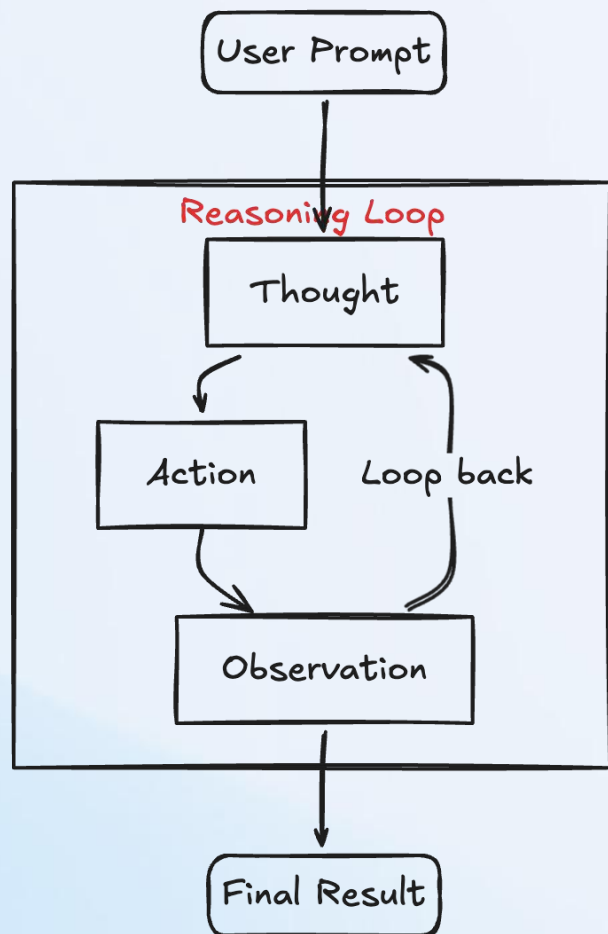
节点状态

✓ 成立

✗ 不成立

? 不确定

Context Engineering

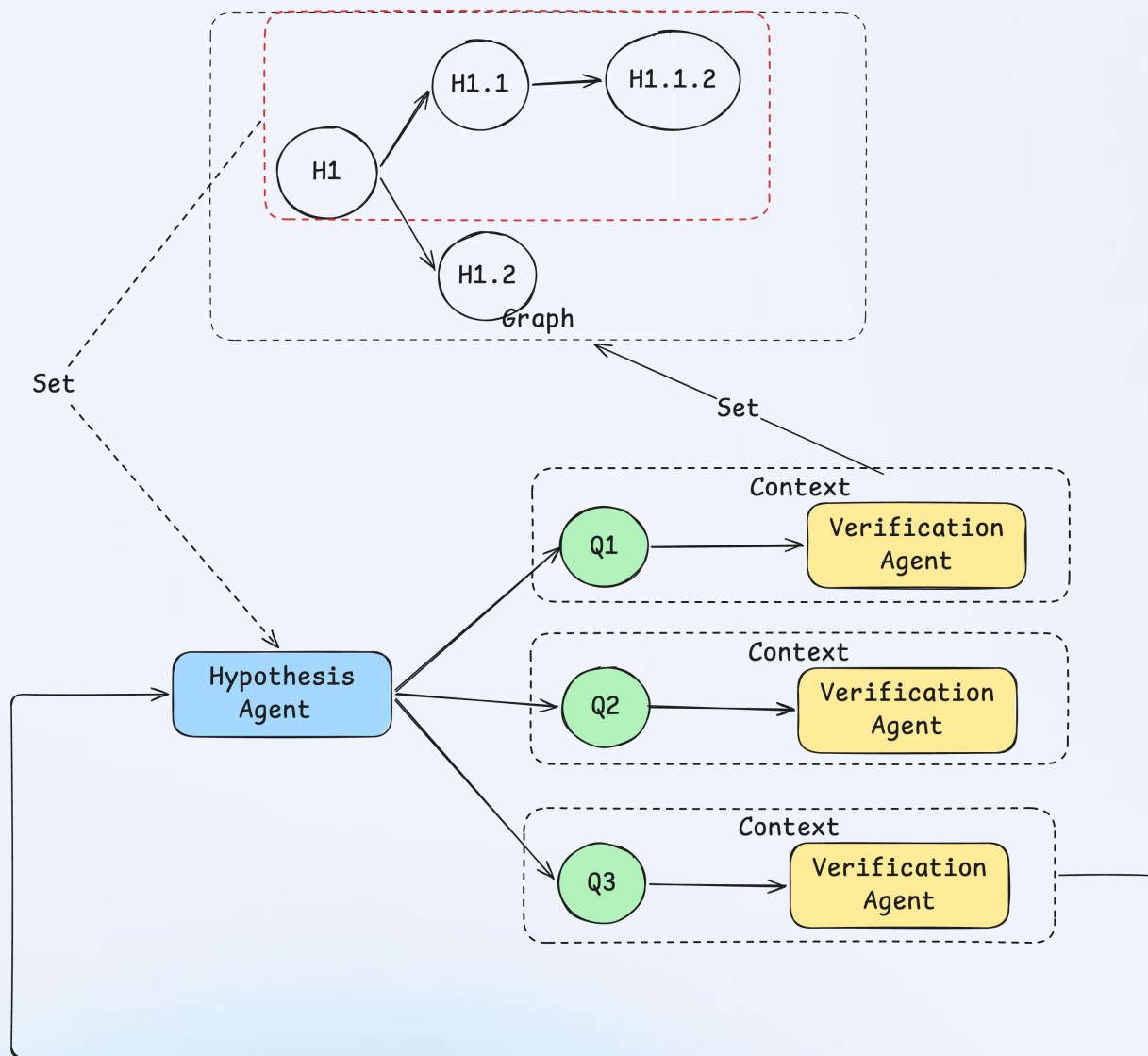


模型容易过早终止推理（着急收尾），压缩会导致遗漏关键细节与排障重点。

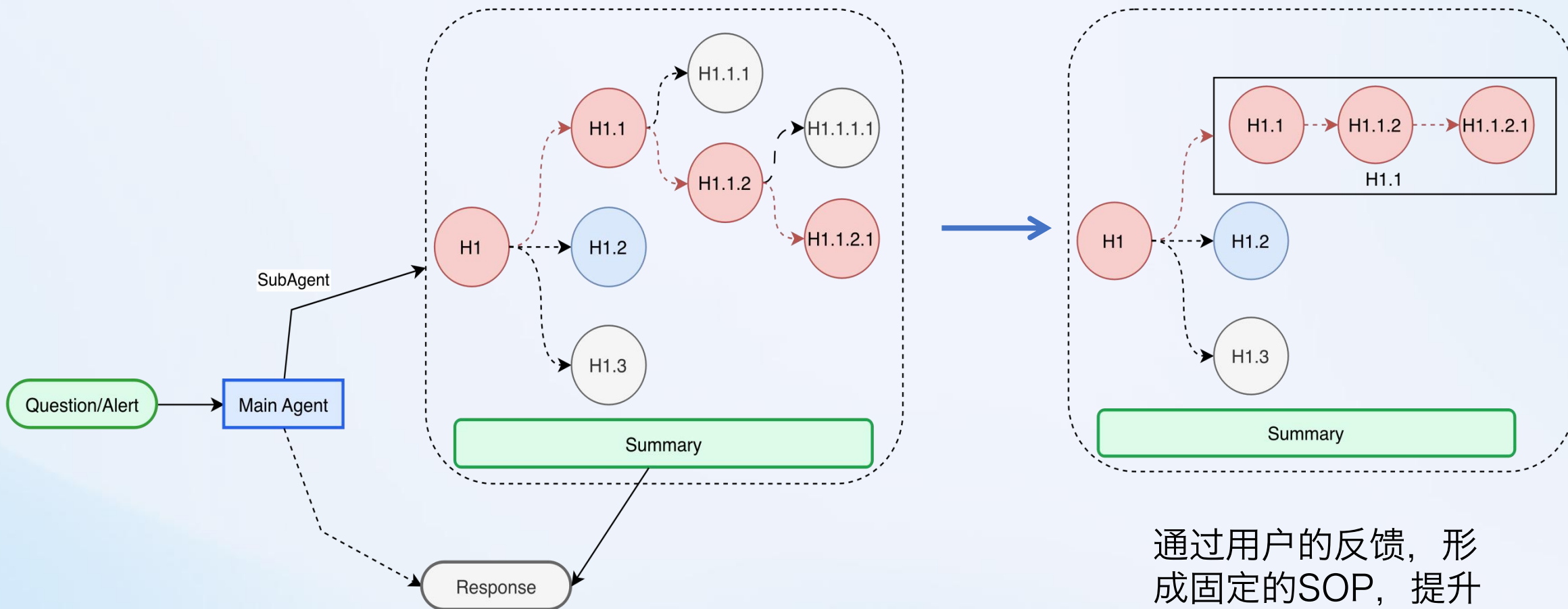
当一个 Context Window 同时承担 ‘假设’ 与 ‘验证’ 任务时（既当裁判又当运动员），产生证实偏差（结果偏乐观）。

Context Engineering

验证时新开单独的上下文窗口，每一次都是“单独提问”。假设时，动态把根到该节点路径注入上下文。



SubAgent + Feedback



通过用户的反馈，形成固定的SOP，提升排查速度。

Memory

Datadog Bits AI SRE



根因是 没有问题，因为 temp-error-generator 容器是一个临时测试容器，它不会导致故障，所以一个忽略掉它

明白了！实际根因是：temp-error-generator 是一个临时测试容器，其崩溃循环是预期行为，不构成故障，应被忽略。

Bits 误判了这个容器的崩溃循环为问题，但实际上它是无害的。

建议改进方向：

- Monitor Runbook — 添加说明：temp-error-generator 是临时测试容器，崩溃循环属正常行为，调查时可忽略
- bits.md — 如果你们还有其他类似的临时/测试容器命名规范（如 temp-*、test-*），可以在这里记录通用规则，让 Bits 在所有调查中都能识别并跳过这类容器

你希望我帮你：

1. 起草 Monitor Runbook 的改进内容？
2. 将测试容器的命名规范写入 bits.md？
3. 两者都做？

👍 🗨 📄

Bits.md

Organization-level instructions that apply to every Bits investigation.

```
1
2  ## 临时/测试容器命名规范
3
4  以下命名模式的容器属于临时测试容器，其崩溃、重启或异常退出均属预期行为，调查时应忽略：
5  - 'temp-*' (如 temp-error-generator)
6  - 'test-*'
7
8  这些容器通常使用基础镜像（如 alpine），运行时间极短，由 Docker restart policy 自动重启，不代表真实故障。
9  调查中遇到此类容器时，请聚焦于其他长期运行的生产容器。
10
11
```

1

bits.md

定义诊断套路、过滤已知噪声、约束执行作用域；确保 Agent 在统一标准下一致性、安全的自动化操作。

2

Monitor Runbook

随告警同步推送，内置故障上下文与标准处置步骤；消除信息差，指导 On-call 工程师快速进入现场并按 SOP 止损。

3

Memories

将人工核实的 RCA（根因）选择性回填至监控器记忆列表；实现“历史排障经验”资产化，确保同类告警触发时知识的精准召回。

Memory

markdown

告警诊断 Skills 定义: 发奖服务接口异常

元数据定义

- **名称 (Name)**: `集群xxx服务接口异常告警`
- **描述 (Description)**: 当接口成功率下降、延迟增加或触发 5xx 错误告警时, 执行此诊

资源映射与定义

服务标识对齐

当匹配到上述告警名称时, Agent 应自动锁定以下资源坐标:

- **Service**: `xxx-api`
- **Namespace**: `xxx-prod`
- **K8s Endpoint**: `service/xxx-api.xxx-prod`

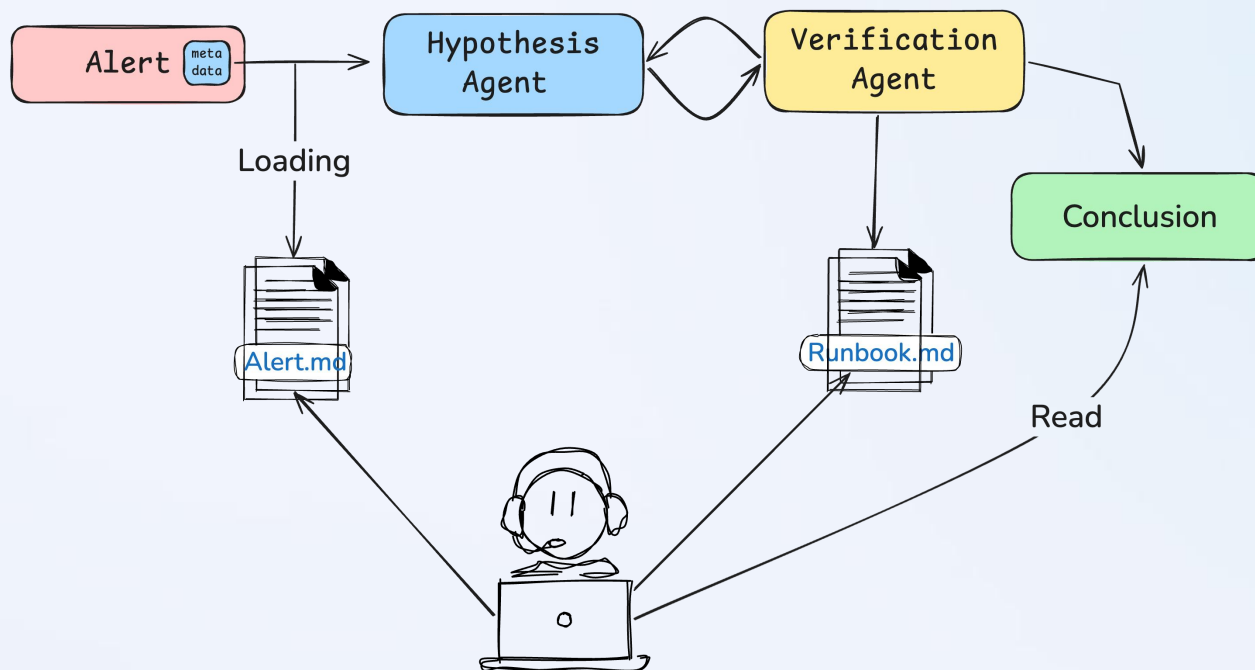
核心下游依赖检查

针对业务特性, Agent 必须优先检查以下特定的下游资产:

1. **数据库 (MySQL)**:
 - **检查项**: 数据库连接池是否耗尽, 是否存在慢 SQL 锁定记录表。
 - **逻辑分析**: 检查 `db.xxx` 的响应时间是否与告警时间点吻合。
2. **缓存 (Redis)**:
 - **检查项**: 库存扣减是否因 Redis 响应慢导致超时。
 - **逻辑分析**: 检查 `cache.xxx-stock` 的命中率和操作延迟。

关键业务指标验证

1. **发奖成功率**: 统计 `/api/v1/xxx` 接口的返回码分布。
2. **库存状态**: 检查配置服务是否下发了错误的零库存标识。



思维链 CoT

risk-platform-backend-svc

微服务 QPS 瞬时跌幅达 50%



假设 (互斥假设)

输入现象: risk-server 告警 - QPS 环比下跌 50%。

输出互斥假设:

H1: 流量上游故障 (外部因素)。

H2: 内部熔断机制触发 (自保因素)。

H3: 节点发生 OOM 重启 (资源因素)。

H4: 数据库写入延迟引发反压 (依赖因素)。

生成验证准则

输入 (Input): H1-H4假设 + 知识库Knowledge + 故障历史。

输出 (Output):

H1: upstream_req_count 流量同步下降 > 40% 则假设成立。

H2: istio_503_count 503 突增则假设成立。

H3: pod_restart_reason 历史记录含有 OOMKilled 则假设成立。

H4: ck_write_latency 延迟超过 2s 且队列积压则假设成立。

事实取证

输入 (Input): 第二步生成的具体查询动作 (PromQL / SQL)。

输出 (Output): 原始事实证据快照。

证据 A: upstream_req_count 为 12k/s, 曲线平稳 (波动 < 2%)。

证据 B: istio_503_count 全链路计数为 0。

证据 C: 检测到 2 次 OOMKilled 事件, 发生在故障时刻。

证据 D: ch_write_p99 为 200ms (处于正常 Baseline)。

结果判定

输入 (Input): 第二步的判定准则 + 第三步的事实证据。

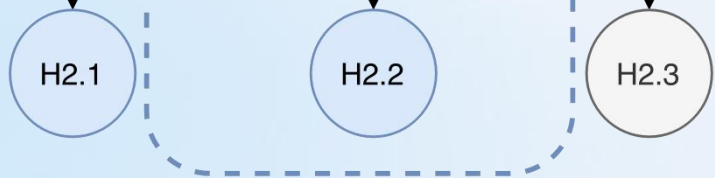
判定逻辑 (Logic Check):

H1判定失败: 证据 A 与准则冲突 (流量未下降)。

H2判定未知: 证据 B 与准则冲突 (无熔断标志)。

H3判定成功: 证据 C 与准则完美匹配。

H4判定失败: 证据 D 显示数据库响应正常。



1

测试输出质量。无论内部实现如何，均有效。

2

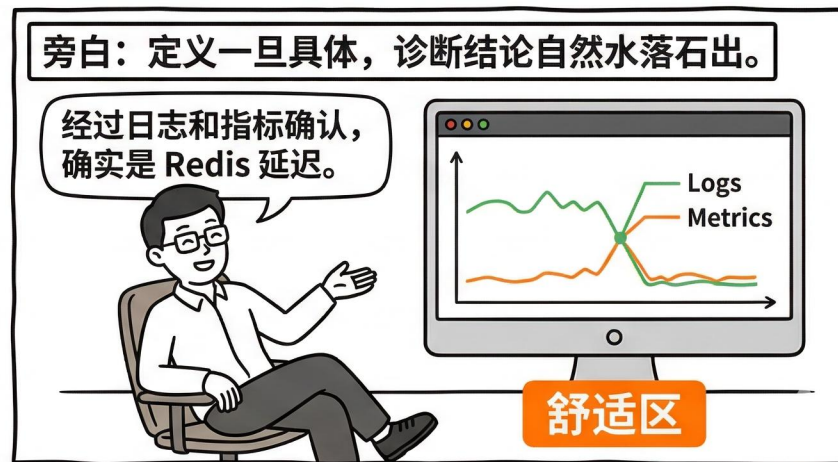
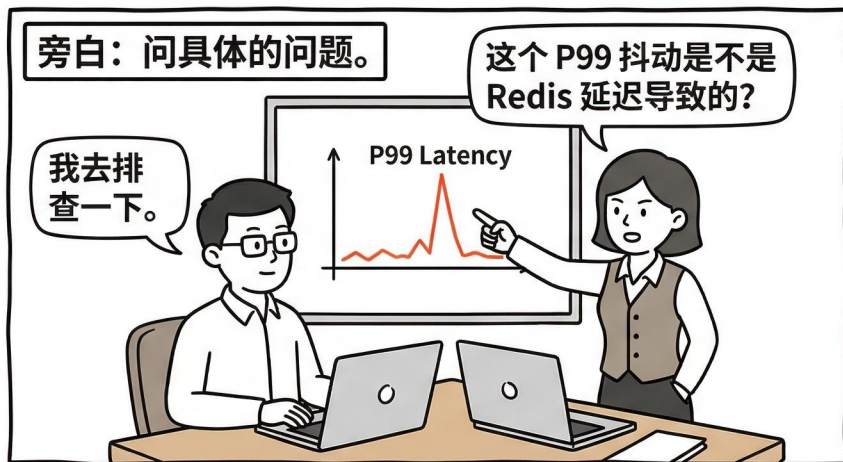
验证代理是否使用了正确的工具序列。



Dataset run verification-agent-tool-benchmark - 2026-04-14T08:28:13.200707Z									
Columns 11/14									
Dataset Item	Trace	Latency	Cost	# tool_precision (api)	# tool_set_exact_m...	Input	Output	Expected	
glh2uvo7ope...		8m 20s	\$0.00	0.0000 	0.0000 	<pre>[{"role": "system", "content": "\n\n你是 SRE 根因分析系统的「验证专家」。你的职责是根据给定的输入，验证根因分析系统是否正确。"}]</pre> Content was truncated.	<pre>{ "selected_tools": [{ "tool_name": "application_metric_latency" }] }</pre>	<pre>{ "role": "system", "content": "\n\n你是 SRE 根因分析系统的「验证专家」。你的职责是根据给定的输入，验证根因分析系统是否正确。"}]</pre> Content was truncated.	
ictywu4lplnb...		8m 35s	\$0.00	1.0000 	0.0000 	<pre>[{"role": "system", "content": "\n\n你是 SRE 根因分析系统的「验证专家」。你的职责是根据给定的输入，验证根因分析系统是否正确。"}]</pre> Content was truncated.	<pre>{ "selected_tools": [{ "tool_name": "application_metric_latency" }] }</pre>	<pre>{ "role": "system", "content": "\n\n你是 SRE 根因分析系统的「验证专家」。你的职责是根据给定的输入，验证根因分析系统是否正确。"}]</pre> Content was truncated.	

过程的可控性远比结果的偶然正确重要。

如何用起来

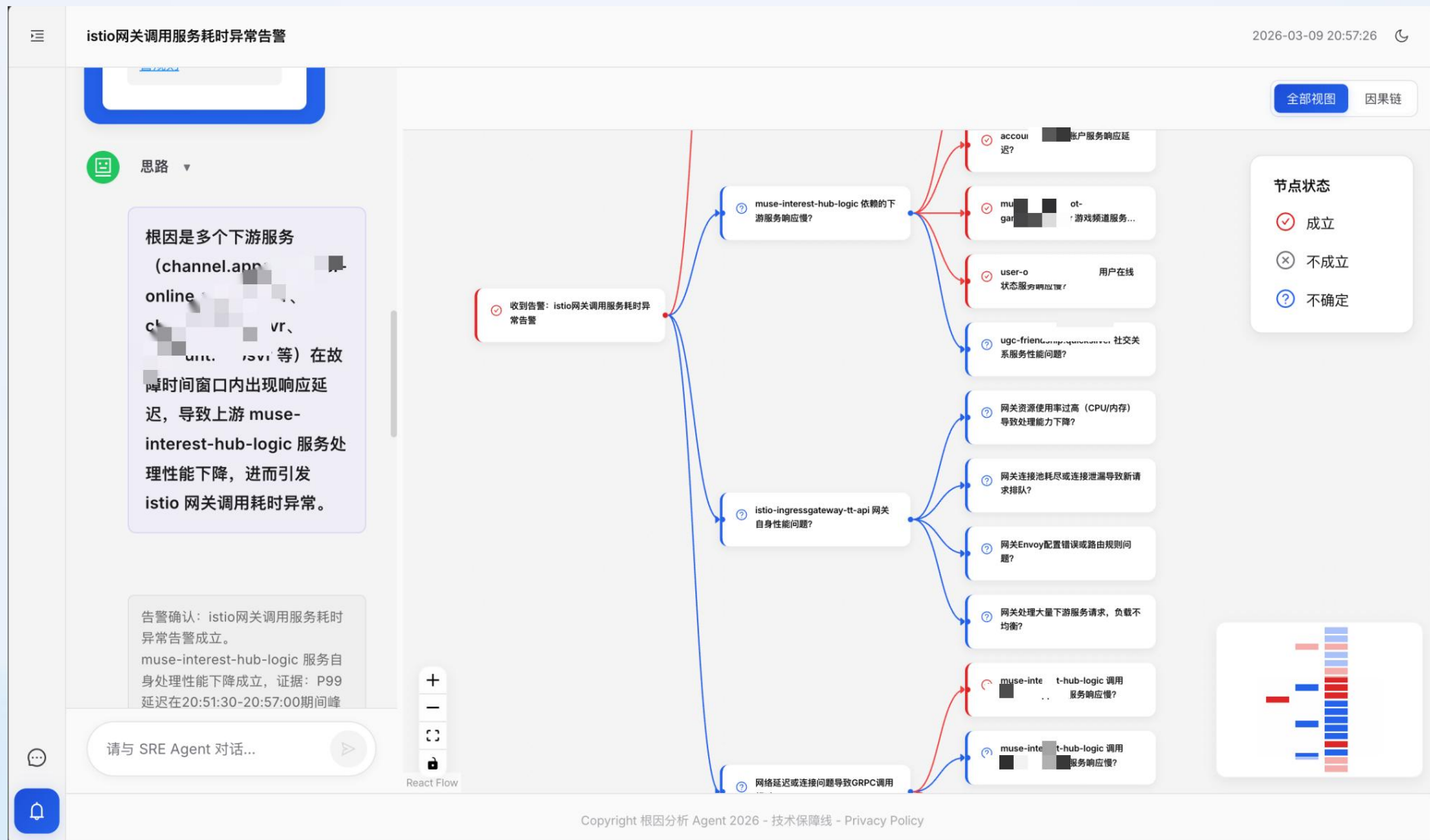


带着具体问题去寻找答案，进而形成固定解法

03

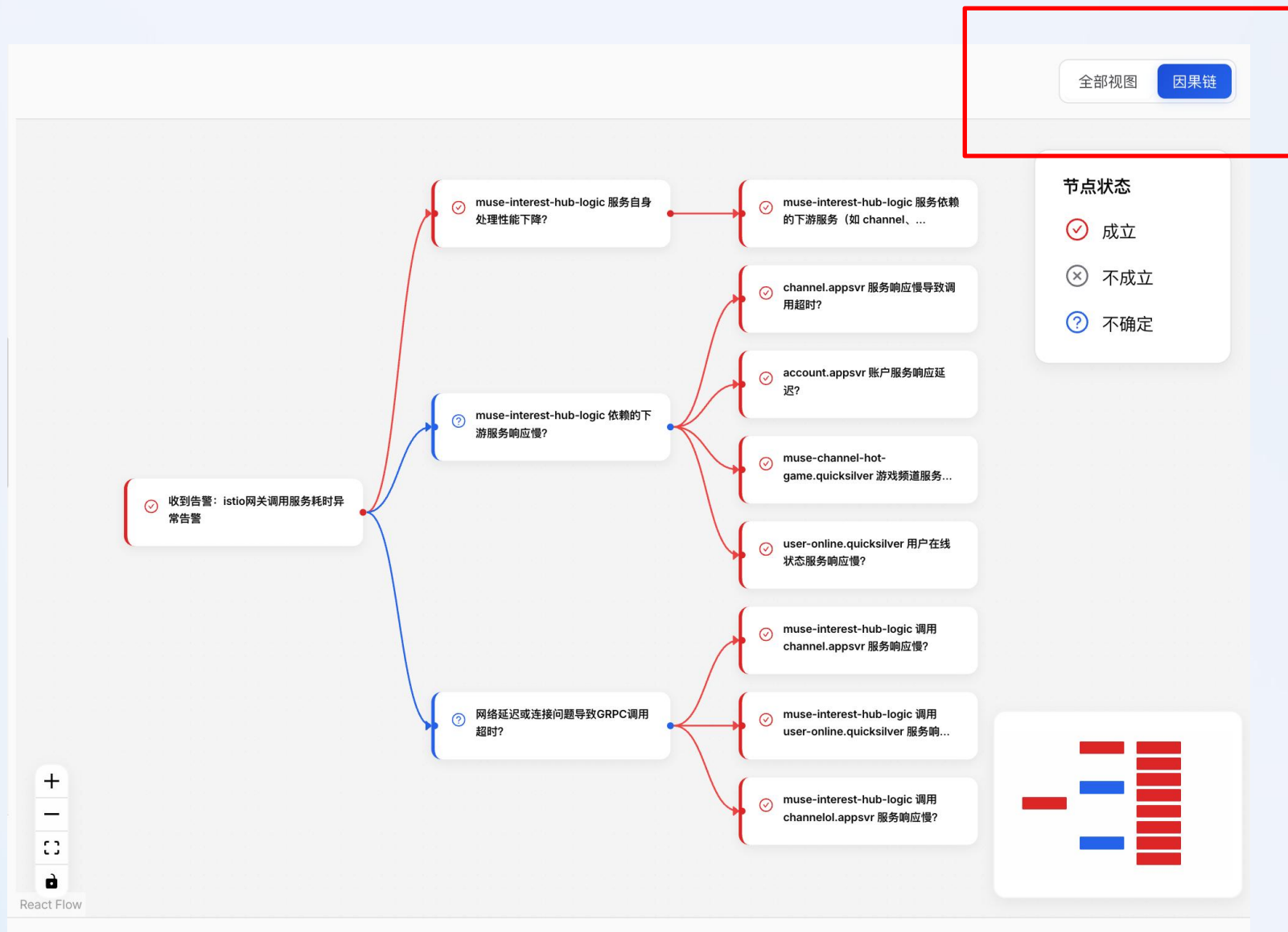
案例展示

实践案例



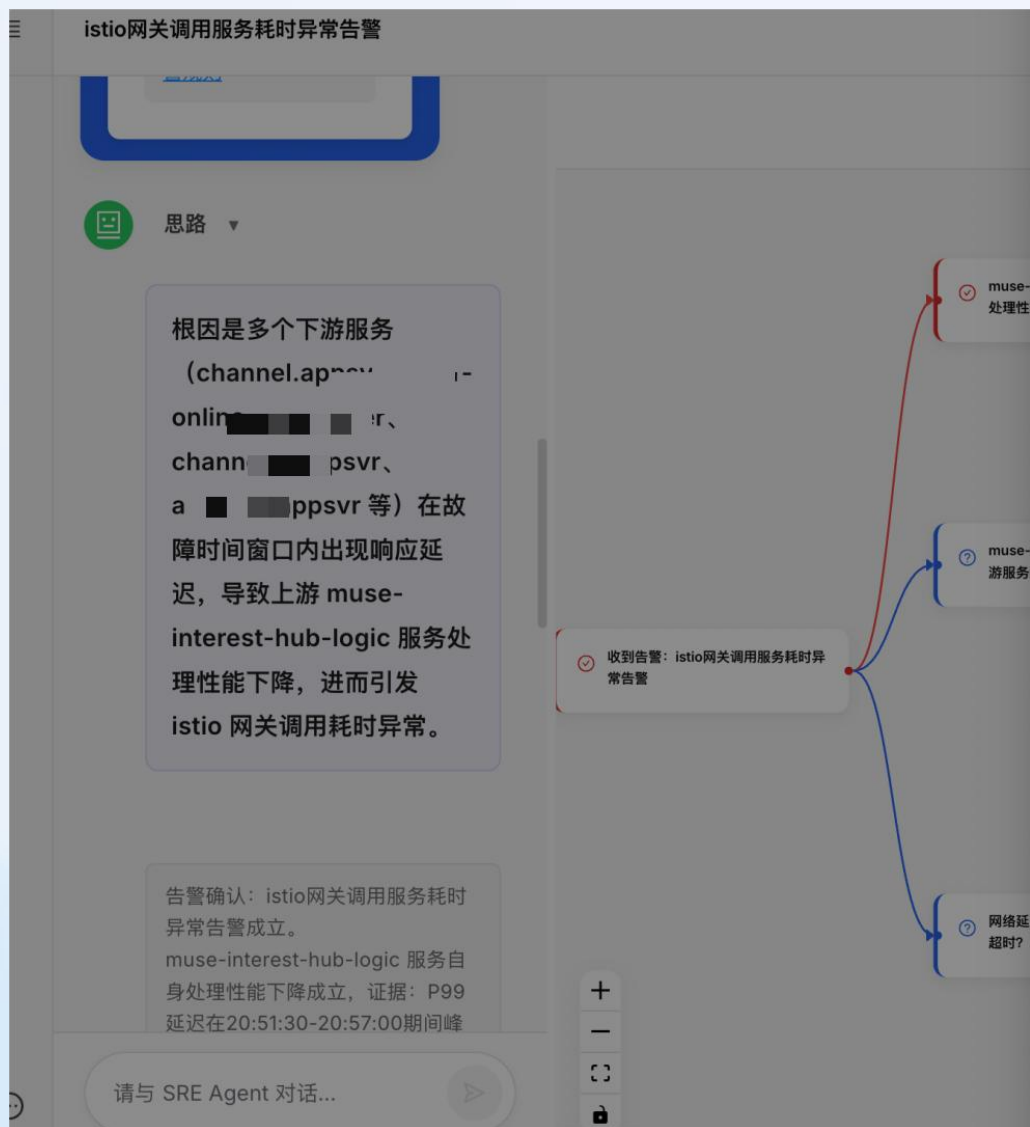


实践案例





实践案例



节点详情: H1.1.3

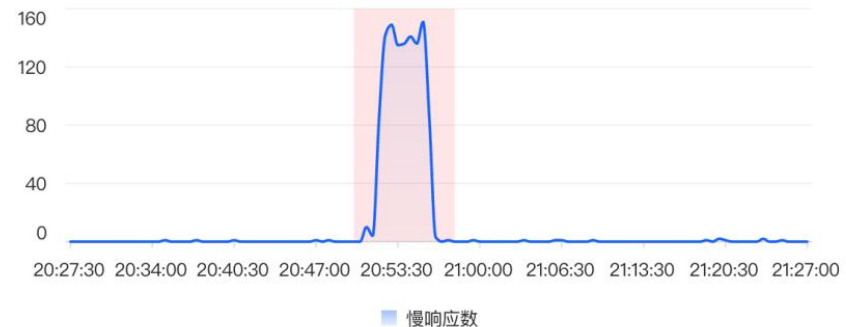
20:28:00 20:35:00 20:42:00 20:49:00 20:56:00 21:03:00 21:10:00 21:17:00 21:24:00

QPS

> 返回内容

Tool: application_metric_slow_response

channel 慢响应数



> 返回内容

Tool: kubernetes_workload_event

> 返回内容

Tool: gateway_application_metric



实践案例

告警内容:

集群 网关[istio-ingressgateway-tt-api 请求服务[muse-interest-hub-logic] 的GRPC调用 P99 持续3分钟>=2500ms (当前值632.16ms)
[查看应用监控](#) [查看告警规则](#)

告警排查

刘至浩

怎么判断是节点故障，有什么证据来判断它是节点故障呢?

某几个pod有问题，对应的pod宿主机有问题

监控中断，宿主机重启

那宿主主机上有什么表象吗?

实践案例





实践案例

istio网关调用服务耗时异常告警

[muse-interest-hub-logic.quicksilver] 的GRPC调用 <u>P99</u> 持续3分钟>=2500ms (当前值54500.50ms)
[查看应用监控](#) [查看告警规则](#)

思路

根因是 muse-interest-hub-logic 服务实例存在严重性能问题，同时其所在的节点出现资源异常 (NotReady)。

收到告警：istio网关调用服务耗时异常告警，P99延迟高达54500.50ms -> muse-interest-hub-logic Pod 所在节点资源异常？成立：节点在 20:54:00 ~ 20:56:00 期间处于 NotReady 状态，内存使用率峰值达89.2% -> muse-interest-hub-logic 服务实例性能问题？成立：服务P99延迟峰值达59.38秒，整体错误率峰值3%，慢响应数峰值4007，6个实例均出现性能恶化

请与 SRE Agent 对话...

React Flow

Copyright 根因分析 Agent

节点详情: H1.2

使用工具: [kubernetes_node_metric_is_not_ready](#)

> 返回内容

使用工具: [kubernetes_node_metric_is_not_ready](#)

> 返回内容

使用工具: [kubernetes_node_metric_is_not_ready](#)

> 返回内容

使用工具: [kubernetes_node_metric_is_not_ready](#)

▼ 返回内容

节点 Prometheus 在 2026-03-09 20:54:00 ~ 2026-03-09 20:56:00 存在 Ready=false (值为 1) 的采样，**该时间段内节点处于 NotReady 异常状态**；

使用工具: [kubernetes_node_metric_is_not_ready](#)

> 返回内容

Tool: [kubernetes_node_metric_is_not_ready](#)

关联 AI 消息 (发起调用)

根据验证协议，我需要检查6个节点的资源使用率，节点IP地址为 10.240.0.1, 10.240.0.2, 10.240.0.3, 10.240.0.4, 10.240.0.5, 10.240.0.6，时间范围是 2026-03-09 20:27:26 到 2026-03-09 21:27:26。

我将并行调用工具检查这些节点的健康状态：



实践案例

[查看应用监控](#) [查看告警规则](#)

[详细](#) [信息](#)

1 条回复

回复 监控: istio网关调用服务耗时异常告警[转发摘要]

istio网关调用服务耗时异常告警

根因分析

根因是 acco 依赖的 MySQL 实例存在慢查询问题，导致下游服务响应延迟，进而引发 muse-interest-hub-logic 服务处理延迟升高，最终触发 istio 网关调用耗时异常告警。

证据链

acco 依赖的 MySQL 实例存在慢查询问题 → 导致 acco 服务响应延迟 → 进而导致 muse-interest-hub-logic 服务处理延迟升高 → 最终触发 istio 网关调用耗时异常告警

- MySQL 实例 mysql-prod-hw-bj-tt-account 慢查询数达到 8.0 个，远超 0.0 个的正常阈值
- account 服务的 P99 延迟在 20:48:00 ~ 20:51:00 期间出现异常，延迟指数为 2.0
- account 服务的 QPS 在 20:50:30 ~ 20:57:30 期间出现异常，异常指数为 3.0
- muse-interest-hub-logic 服务的 P99 延迟在 20:51:00 ~ 20:57:00 期间出现异常，延迟峰值达 59.38 秒
- muse-interest-hub-logic 服务的错误率在 20:50:30 ~ 20:55:00 期间异常升高，达到 3.0%
- muse-interest-hub-logic 服务的慢响应数在 20:50:30 ~ 20:57:00 期间显著异常，部分实例峰值达 3988 次/分钟
- 下游依赖服务 channel 的 P99 延迟在 20:46:00 ~ 20:51:30 期间出现异常，延迟指标达 3.0 倍异常值

✓ muse-interest-hub-logic 服务本身处理延迟高，可能是CPU/内存资源不足？ | H1.1

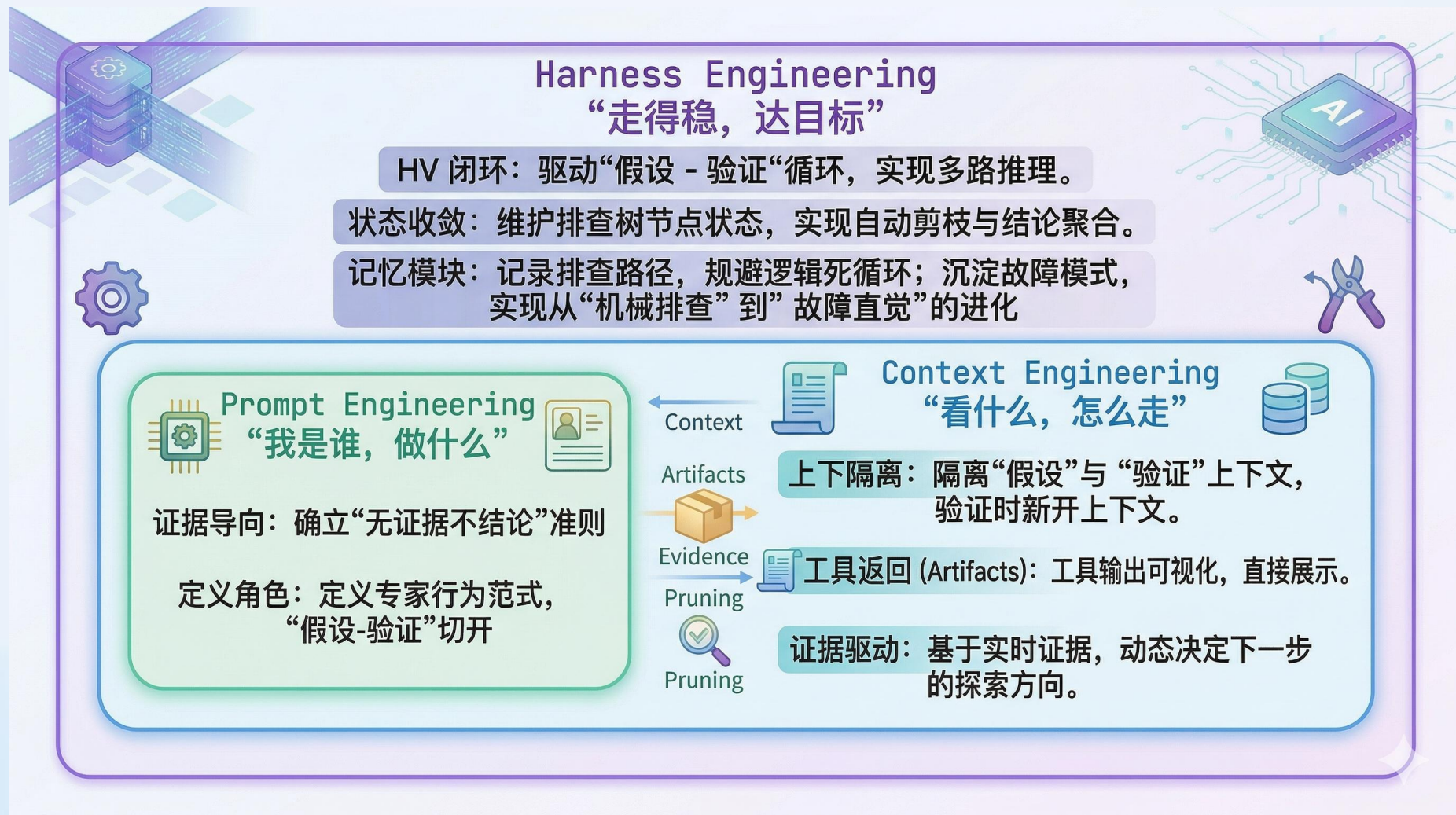
查看详情



04

思考与总结

Agent 演进



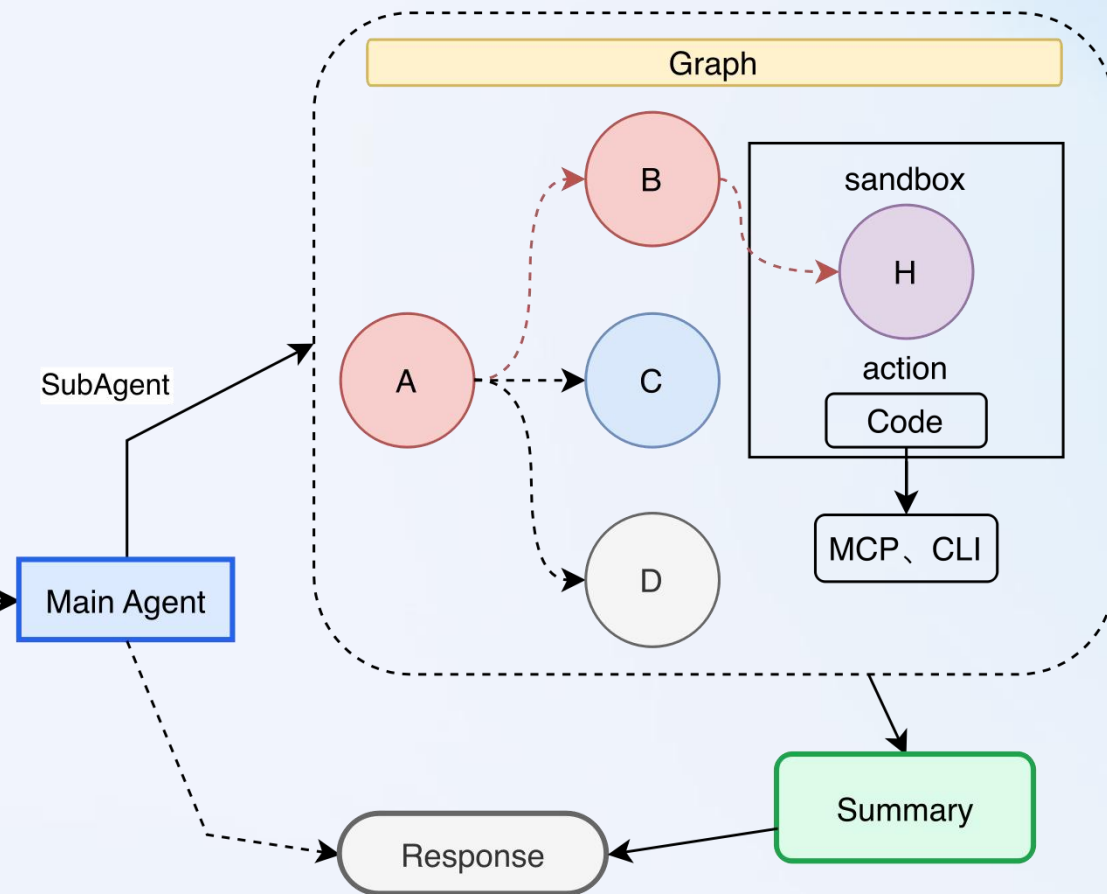
AI Agent: 标准化运维的加速器



05 未来展望

自主决策执行

权限、安全、受控执行、回滚



根因定位层级



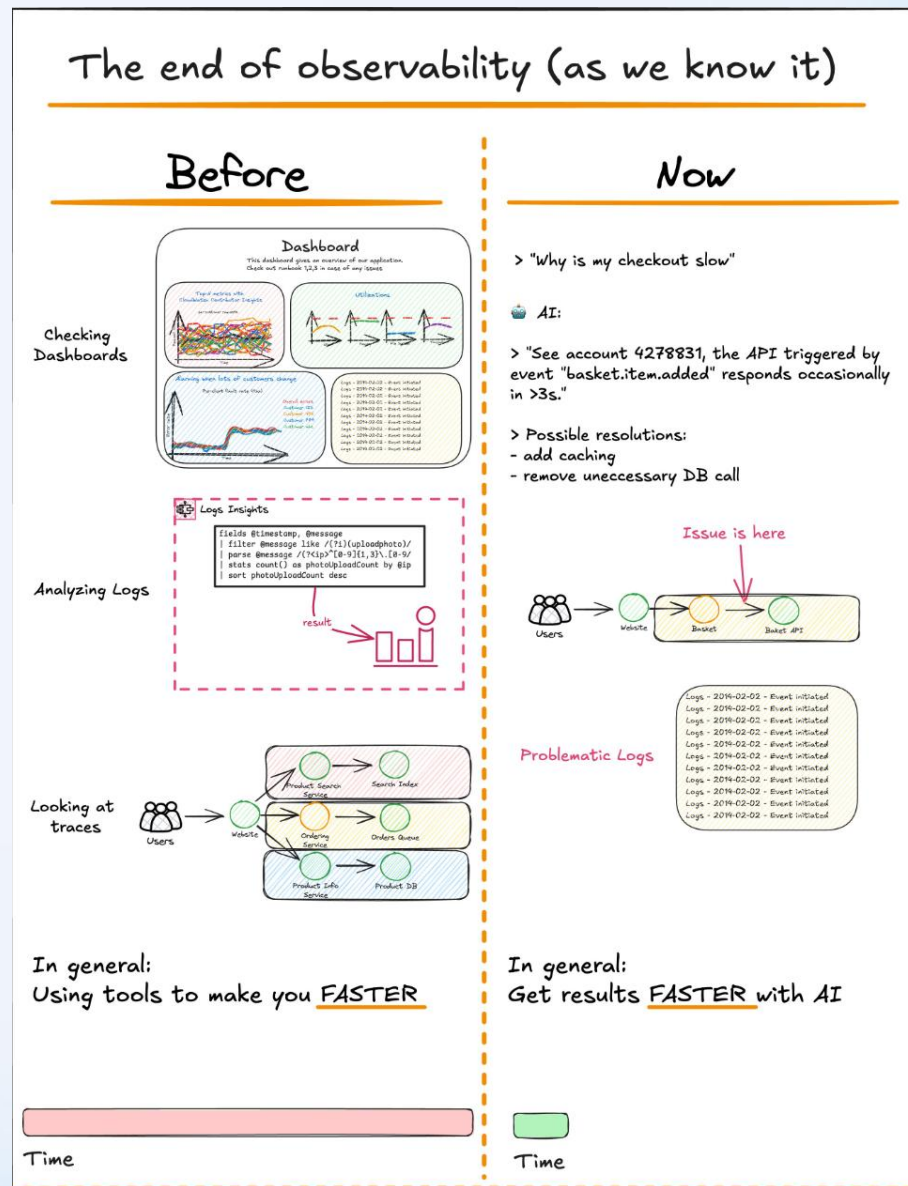
Agent-Native

“AI First” 系统应优先考虑对 Agent 友好

过去监控是给人看的，未来监控要给 Agent “看”（提供语义接口 Semantic API）。

“可观测性的语义化”

当你的 logging/tracing/metrics/events 能让 Agent 读起来毫无阻力时，真正的智能运维才会到来。



引用:

https://x.com/sandro_vol/status/1993727421867294756

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

